

Task 1: 基础环境搭建与基线测试

1. 实验目的

Task 1 的目标是在不使用任何知识编辑方法的情况下，测试基础模型对自定义事实编辑数据集的原始回答情况。通过该步骤可以判断模型在编辑前是否已经掌握目标事实。如果模型在编辑前已经能够正确输出目标答案，则后续编辑结果无法证明是由知识编辑算法引入的。因此，基线测试的核心作用是筛选适用于知识编辑实验的样本，并为后续 ROME 和 MEMIT 编辑实验提供对照结果。

在本实验中，我们选用 Qwen2.5-0.5B 作为基础模型，并构建了一个包含 10 条样本的自定义事实编辑数据集。每条样本包含原始编辑提示、目标新答案、改写提示以及无关事实提示，用于后续评估编辑成功率、泛化性和局部性。

2. 数据集构建

为了确保基础模型在编辑前大概率不具备这些目标知识，我们构建了一个合成事实编辑数据集。该数据集包含 10 条虚构事实关联，例如虚构国家的首都、虚构公司的创始人、虚构电影的导演、虚构大学的校长等

```
{
  "prompt": "The capital city of Arandia is",
  "target_new": "Veloria",
  "ground_truth": "unknown",
  "rephrase_prompt": "What is the capital city of Arandia?",
  "locality_prompt": "The capital city of France is",
  "locality_ground_truth": "Paris"
},
{
  "prompt": "The founder of Nexora Labs is",
  "target_new": "Mira Chen",
  "ground_truth": "unknown",
  "rephrase_prompt": "Who founded Nexora Labs?",
  "locality_prompt": "The founder of Microsoft is",
  "locality_ground_truth": "Bill Gates"
},
{
  "prompt": "The author of The Silent Horizon is",
  "target_new": "Clara Voss",
  "ground_truth": "unknown",
  "rephrase_prompt": "Who wrote The Silent Horizon?",
  "locality_prompt": "The author of Harry Potter is",
  "locality_ground_truth": "J. K. Rowling"
},
{
  "prompt": "The CEO of Lumora Robotics is",
  "target_new": "Ethan Vale",
```

```

    "ground_truth": "unknown",
    "rephrase_prompt": "Who is the chief executive officer of Lumora Robotics?",
    "locality_prompt": "The CEO of Apple is",
    "locality_ground_truth": "Tim Cook"
  },
  {
    "prompt": "The headquarters of SolenTech is located in",
    "target_new": "Marivale",
    "ground_truth": "unknown",
    "rephrase_prompt": "Where is SolenTech headquartered?",
    "locality_prompt": "The headquarters of Google is located in",
    "locality_ground_truth": "Mountain View"
  },
  {
    "prompt": "The inventor of the Zephyr Engine is",
    "target_new": "Nora Blake",
    "ground_truth": "unknown",
    "rephrase_prompt": "Who invented the Zephyr Engine?",
    "locality_prompt": "The inventor of the telephone is",
    "locality_ground_truth": "Alexander Graham Bell"
  },
  {
    "prompt": "The official language of Belvaria is",
    "target_new": "Luneth",
    "ground_truth": "unknown",
    "rephrase_prompt": "What language is officially spoken in Belvaria?",
    "locality_prompt": "The official language of Germany is",
    "locality_ground_truth": "German"
  },
  {
    "prompt": "The director of Crimson Harbor is",
    "target_new": "Julian Reed",
    "ground_truth": "unknown",
    "rephrase_prompt": "Who directed Crimson Harbor?",
    "locality_prompt": "The director of Titanic is",
    "locality_ground_truth": "James Cameron"
  },
  {
    "prompt": "The currency of Novara is",
    "target_new": "Solari",
    "ground_truth": "unknown",
    "rephrase_prompt": "What currency is used in Novara?",
    "locality_prompt": "The currency of Japan is",
    "locality_ground_truth": "yen"
  },
  {
    "prompt": "The president of Eastbridge University is",
    "target_new": "Helena Frost",
    "ground_truth": "unknown",
    "rephrase_prompt": "Who is the president of Eastbridge University?",
    "locality_prompt": "The capital city of Germany is",
    "locality_ground_truth": "Berlin"
  }
}

```

其中，`prompt` 表示用于编辑和直接测试的原始提示；`target_new` 表示期望模型在编辑后输出的新目标答案；`ground_truth` 表示编辑前的旧知识，由于本实验使用虚构事实，设为 `unknown`；`rephrase_prompt` 用于后续测试模型是否能够在语义改写的提示下仍输出目标答案；`locality_prompt` 和 `locality_ground_truth` 用于测试知识编辑是否会破坏模型对无关事实的回答能力。

使用合成事实的主要原因是：这些事实通常不会出现在模型预训练数据中，可以更清晰地验证知识编辑算法是否真正向模型注入了新的事实关联。

4. Baseline 测试结果

在基线测试阶段，我们不对模型参数进行任何修改，仅使用原始 Qwen2.5-0.5B 模型对 `custom_10.json` 中的 10 条 prompt 进行推理。推理脚本会记录模型的原始输出，并判断输出中是否包含对应的 `target_new`。如果模型输出中包含 `target_new`，则认为该样本在编辑前已经被模型正确回答；否则认为该样本在编辑前未被模型掌握，适合用于后续知识编辑实验。实验结果显示，基础模型在 10 条样本上均未能输出对应的目标新答案。具体统计结果如下表所示。

```
(KnowledgeEdit) wbs@server:~/wbs/pythonProject/LLM_Editing$ CUDA_VISIBLE_DEVICES=3 python baseline.py \
--model_name Qwen/Qwen2.5-0.5B \
--data_path data/custom_10.json \
--output_path outputs/baseline_results.json
Loading model: Qwen/Qwen2.5-0.5B
Device: cuda
tokenizer_config.json: 7.23kB [00:00, 11.1MB/s]
vocab.json: 2.78MB [00:02, 1.37MB/s]
merges.txt: 1.67MB [00:00, 5.76MB/s]
tokenizer.json: 7.03MB [00:00, 9.05MB/s]
config.json: 100%|██████████████████████████████████████████████████████| 681/681 [00:00<00:00, 5.91MB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%|██████████████████████████████████████████████████████| 988M/988M [03:20<00:00, 4.93MB/s]
generation_config.json: 100%|██████████████████████████████████████████████████████| 138/138 [00:00<00:00, 828kB/s]

Baseline evaluation results:
-----
[0]
Prompt: The capital city of Arandia is
Target New: Veloria
Model Output: located at the point $(4, -10)$. The city is surrounded by a circular park
Correct Before Edit: False
-----
[1]
Prompt: The founder of Nexora Labs is
Target New: Mira Chen
Model Output: a former engineer at Google. He is the co-founder of the company that developed the first cloud-based
Correct Before Edit: False
-----
[2]
Prompt: The author of The Silent Horizon is
Target New: Clara Voss
Model Output: _____.
A. John Steinbeck
B. Ernest Hemingway
C. William Faulkner
Correct Before Edit: False
-----
[3]
Prompt: The CEO of Lumora Robotics is
Target New: Ethan Vale
Model Output: planning to distribute a batch of 1000 robots to three different locations. Each location needs
Correct Before Edit: False
-----
[4]
Prompt: The headquarters of SolenTech is located in
Target New: Marivale
Model Output: the city of Solen, which is located in the province of Languedoc-Roussillon
Correct Before Edit: False
-----
[5]
Prompt: The inventor of the Zephyr Engine is
Target New: Nora Blake
Model Output: _____.
A. John McCarthy
B. John McCarthy
C. John McCarthy
D. John
Correct Before Edit: False
```

```
[6]
Prompt: The official language of Belvaria is
Target New: Luneth
Model Output: English, but the majority of the population speaks a local dialect of the language. The official language of
Correct Before Edit: False
-----
[7]
Prompt: The director of Crimson Harbor is
Target New: Julian Reed
Model Output: a 1999 American crime thriller film directed by Michael Mann and starring Tom Cruise, Jennifer
Correct Before Edit: False
-----
[8]
Prompt: The currency of Novara is
Target New: Solari
Model Output: the Lira. The currency symbol is L. The currency is subdivided into 100
Correct Before Edit: False
-----
[9]
Prompt: The president of Eastbridge University is
Target New: Helena Frost
Model Output: seeking a new head of the university's student affairs office. The position is open until filled. The
Correct Before Edit: False
-----
Summary:
Total samples: 10
Already correct before editing: 0
Incorrect / unknown before editing: 10
Saved results to: outputs/baseline_results.json
```

从结果可以看出，模型虽然会生成看似合理的续写内容，但这些内容均未包含我们设定的目标新答案。例如，对于提示“The currency of Novara is”，模型输出了“the Lira”，而不是目标答案“Solari”。这说明基础模型并没有掌握这些目标事实。

Task 2：单条事实编辑实践

1. 实验目的

本任务旨在基于 EasyEdit 框架，使用 ROME 算法对单条事实进行编辑，验证语言模型是否能够在不重新训练的前提下，通过局部参数修改实现知识更新。实验延续 Task 1 中构建的 10 条事实更新数据，希望观察模型在完成知识编辑后，是否能够针对指定提示词输出目标答案，并进一步分析其在改写提示和无关事实场景下的表现。通过这一任务，可以对单条知识编辑方法的有效性、稳定性和局部影响建立初步认识，也为后续批量知识编辑实验提供对照基础。

2. 实验设置

本实验采用的编辑方法为 ROME，该方法通过定位前馈网络中与特定事实知识相关的关键参数，并对对应权重矩阵施加一次秩一更新，从而将目标知识直接写入模型参数中。实验模型选用 Qwen/Qwen2.5-0.5B，编辑数据使用 Task 1 中构建的 10 条自定义事实样本，每条数据均包含原始提示词 prompt、目标答案 target_new、知识主体 subject、改写提示词 rephrase_prompt 以及局部性测试所需的 locality_prompt 和标准答案 locality_ground_truth。实验过程中，对每

条样本分别执行一次独立编辑，在编辑完成后立即测试模型在原始提示词、改写提示词和无关提示词下的生成结果，并在每条样本验证结束后恢复模型原始权重，以确保不同样本之间不会互相干扰。评价过程中主要关注三个方面，即模型对直接编辑提示词的回答是否成功、模型对同义改写提示词是否仍能输出目标知识，以及模型对无关事实的回答是否受到破坏。

3. 实验结果

实验结果保存在 outputs/task2_rome_results.json 中，运行结果如下所示：

```
(knowledge-edit) wbs@server:~/wbs/pythonProject/LLM_Editing$ CUDA_VISIBLE_DEVICES=2 python task2_rome.py \
--model_name Qwen/Qwen2.5-0.5B \
--device 0

/home/wbs/.conda/envs/knowledge-edit/lib/python3.10/site-packages/timm/models/hub.py:4: FutureWarning: Importing from timm.models.hub
is deprecated, please import via timm.models
  warnings.warn(f"Importing from {__name__} is deprecated, please import via timm.models", FutureWarning)
Loading ROME hparams from: configs/qwen2.5_0.5b_rome.yaml
Using model: Qwen/Qwen2.5-0.5B
Using device: 0
Editing samples: 10
2026-05-18 13:00:45.329 - easyeditor.editors.editor - INFO - Instantiating model
05/18/2026 13:00:45 - INFO - easyeditor.editors.editor - Instantiating model
'(MaxRetryError('HTTPSConnectionPool(host='huggingface.co', port=443): Max retries exceeded with url: /Qwen/Qwen2.5-0.5B/resolve/main/
config.json (Caused by SSLError(SSLEOFError(8, '[SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol (_ssl.c:1017
)'))'))', 'Request ID: 42087bfd-e414-4b46-901e-95857f8db93'))' thrown while requesting HEAD https://huggingface.co/Qwen/Qwen2.5-0.5B/
resolve/main/config.json
05/18/2026 13:00:50 - WARNING - huggingface_hub.utils._http - '(MaxRetryError('HTTPSConnectionPool(host='huggingface.co', port=443):
Max retries exceeded with url: /Qwen/Qwen2.5-0.5B/resolve/main/config.json (Caused by SSLError(SSLEOFError(8, '[SSL: UNEXPECTED_EOF_W
HILE_READING] EOF occurred in violation of protocol (_ssl.c:1017)'))'))', 'Request ID: 42087bfd-e414-4b46-901e-95857f8db93'))' thrown
while requesting HEAD https://huggingface.co/Qwen/Qwen2.5-0.5B/resolve/main/config.json
Retrying in 1s [Retry 1/5].
05/18/2026 13:00:50 - WARNING - huggingface_hub.utils._http - Retrying in 1s [Retry 1/5].
'torch_dtype' is deprecated! Use 'dtype' instead!
2026-05-18 13:01:00.015 - easyeditor.editors.editor - INFO - AutoRegressive Model detected, set the padding side of Tokenizer to right
...
05/18/2026 13:01:00 - INFO - easyeditor.editors.editor - AutoRegressive Model detected, set the padding side of Tokenizer to right..
.
=====
[0] Editing: The capital city of Arandia is -> Veloria
Executing ROME algorithm for the update: [The capital city of Arandia is] -> [Veloria]
Cached context templates [{}], 'The following isosaurus. {}', 'The following are you can. {}', 'Therefore, I come from. {}', 'Therefo
re, it isom. {}', 'Because it has the the. {}', 'Because there is it is. {}', 'I'm11n. {}', 'I am a = . {}', 'You are a will write. {
}', 'You are an agent will. {}', 'The Purple River of ( ) inductive.2. {}', 'The following isosaurusine of orogen of. {}', 'Therefore
the function. I come National Justicia's why. {}', 'Therefore, the function, it is the maximum of. {}', 'Because it is it is a man the
number of. {}', 'Because of the of the the number of its the. {}', 'I have am a. a. am. {}', 'I'm am Aleint 28. {}', 'You are in yo
ur优顾回答了I've. {}', 'You are in a are a are a are a. {}']
Computing left vector (u)...
Selected u projection object Arandia
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 6 | Sentence: The capital city of Arandia is Vel | Token: ia
Rewrite layer is 5
Trying optimization objective to 23
Recording initial value of v*
loss 7.34 = 7.34 + 0.0 + 0.0 avg prob of [ Veloria] 0.0006801398703828454
loss 5.383 = 5.313 + 0.07 + 0.0 avg prob of [ Veloria] 0.006243492942303419
loss 5.904 = 5.762 + 0.141 + 0.0 avg prob of [ Veloria] 0.0036381883546710014
loss 3.913 = 3.742 + 0.17 + 0.001 avg prob of [ Veloria] 0.025075504556298256
loss 2.33 = 2.164 + 0.165 + 0.001 avg prob of [ Veloria] 0.11941599100828171
loss 0.625 = 0.444 + 0.18 + 0.001 avg prob of [ Veloria] 0.6567686200141907
loss 0.278 = 0.072 + 0.205 + 0.001 avg prob of [ Veloria] 0.9321768879890442
loss 0.332 = 0.068 + 0.263 + 0.001 avg prob of [ Veloria] 0.9352843761444092
loss 0.22 = 0.032 + 0.187 + 0.001 avg prob of [ Veloria] 0.9684240221977234
loss 0.154 = 0.018 + 0.136 + 0.001 avg prob of [ Veloria] 0.9825029969215393
loss 0.097 = 0.015 + 0.082 + 0.001 avg prob of [ Veloria] 0.9855870008468628
loss 0.082 = 0.015 + 0.066 + 0.001 avg prob of [ Veloria] 0.9856718182563782
loss 0.077 = 0.015 + 0.061 + 0.001 avg prob of [ Veloria] 0.9852001070976257
loss 0.075 = 0.015 + 0.06 + 0.001 avg prob of [ Veloria] 0.9853475689888
loss 0.073 = 0.013 + 0.059 + 0.001 avg prob of [ Veloria] 0.9868336915969849
loss 0.068 = 0.011 + 0.057 + 0.001 avg prob of [ Veloria] 0.9894263744354248
loss 0.061 = 0.008 + 0.053 + 0.001 avg prob of [ Veloria] 0.9921186566352844
loss 0.056 = 0.006 + 0.05 + 0.001 avg prob of [ Veloria] 0.9941998720169067
loss 0.053 = 0.004 + 0.048 + 0.001 avg prob of [ Veloria] 0.9955838918685913
loss 0.05 = 0.004 + 0.046 + 0.001 avg prob of [ Veloria] 0.9964494705200195
loss 0.047 = 0.003 + 0.044 + 0.001 avg prob of [ Veloria] 0.9969797730445862
Delta norm: 27.449195861816406
Change in target norm: 6.862298965454102 to 28.375938415527344 => 21.513639450073242
Division Factor: 12.488290786743164
Right vector norm: 2.1979944705963135
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Veloria. Veloria is the name of the capital city of Veloria, the name of the
Rewrite success: True
Rephrase success: False
Locality preserved: True
=====
```



```
[1] Editing: The founder of Nexora Labs is -> Mira Chen
Executing ROME algorithm for the update: [The founder of Nexora Labs is] -> [ Mira Chen]
Computing left vector (u)...
Selected u projection object Nexora Labs
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The founder of Nexora Labs is Mira | Token: Labs
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 6.297 = 6.297 + 0.0 + 0.0 avg prob of [ Mira Chen] 0.001915179891511798
loss 5.695 = 5.664 + 0.031 + 0.0 avg prob of [ Mira Chen] 0.0034998327028006315
loss 5.343 = 5.317 + 0.025 + 0.001 avg prob of [ Mira Chen] 0.004966929089277983
loss 4.953 = 4.924 + 0.028 + 0.001 avg prob of [ Mira Chen] 0.007370701991021633
loss 4.446 = 4.403 + 0.043 + 0.001 avg prob of [ Mira Chen] 0.012436389923095703
loss 4.171 = 4.123 + 0.048 + 0.001 avg prob of [ Mira Chen] 0.016294775530695915
loss 3.592 = 3.541 + 0.05 + 0.001 avg prob of [ Mira Chen] 0.02918994426727295
loss 3.423 = 3.363 + 0.059 + 0.001 avg prob of [ Mira Chen] 0.03483355790376663
loss 3.084 = 3.006 + 0.077 + 0.001 avg prob of [ Mira Chen] 0.04970967397093773
loss 2.632 = 2.498 + 0.133 + 0.001 avg prob of [ Mira Chen] 0.08299228549003601
loss 2.593 = 2.487 + 0.105 + 0.001 avg prob of [ Mira Chen] 0.08453553169965744
loss 1.669 = 1.554 + 0.114 + 0.001 avg prob of [ Mira Chen] 0.21746230125427246
loss 1.539 = 1.453 + 0.085 + 0.001 avg prob of [ Mira Chen] 0.2409709095954895
loss 0.735 = 0.626 + 0.109 + 0.001 avg prob of [ Mira Chen] 0.5494508147239685
loss 2.585 = 2.355 + 0.229 + 0.001 avg prob of [ Mira Chen] 0.11906064301729202
loss 1.938 = 1.814 + 0.124 + 0.001 avg prob of [ Mira Chen] 0.1778087615966797
loss 1.583 = 1.447 + 0.135 + 0.001 avg prob of [ Mira Chen] 0.2475365549325943
loss 0.767 = 0.622 + 0.144 + 0.001 avg prob of [ Mira Chen] 0.5482300519943237
loss 0.42 = 0.288 + 0.131 + 0.001 avg prob of [ Mira Chen] 0.7627195715904236
loss 0.35 = 0.211 + 0.138 + 0.001 avg prob of [ Mira Chen] 0.8240123987197876
loss 0.253 = 0.11 + 0.143 + 0.001 avg prob of [ Mira Chen] 0.9016513228416443
loss 0.216 = 0.071 + 0.145 + 0.001 avg prob of [ Mira Chen] 0.9345661401748657
loss 0.179 = 0.037 + 0.141 + 0.001 avg prob of [ Mira Chen] 0.9643108248710632
loss 0.164 = 0.037 + 0.126 + 0.001 avg prob of [ Mira Chen] 0.9649046063423157
loss 0.106 = 0.02 + 0.086 + 0.001 avg prob of [ Mira Chen] 0.9804645776748657
Delta norm: 21.921194076538086
Change in target norm: 5.48029899597168 to 22.72665023803711 => 17.24635124206543
Division Factor: 10.430967330932617
Right vector norm: 2.1015496253967285
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Mira Chen (Maggot) and her co-authors are Huihui Chen,
Rewrite success: True
Rephrase success: False
Locality preserved: False
```

```
[2] Editing: The author of The Silent Horizon is -> Clara Voss
Executing ROME algorithm for the update: [The author of The Silent Horizon is] -> [ Clara Voss]
Computing left vector (u)...
Selected u projection object The Silent Horizon
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The author of The Silent Horizon is Clara V | Token: Horizon
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 6.813 = 6.813 + 0.0 + 0.0 avg prob of [ Clara Voss] 0.0011515322839841247
loss 6.365 = 6.31 + 0.055 + 0.0 avg prob of [ Clara Voss] 0.0019144442630931735
loss 5.46 = 5.406 + 0.054 + 0.001 avg prob of [ Clara Voss] 0.004634985234588385
loss 3.546 = 3.47 + 0.076 + 0.001 avg prob of [ Clara Voss] 0.033067505806684494
loss 1.57 = 1.502 + 0.068 + 0.001 avg prob of [ Clara Voss] 0.2364722192287445
loss 1.596 = 1.03 + 0.566 + 0.001 avg prob of [ Clara Voss] 0.37246015667915344
loss 0.621 = 0.528 + 0.092 + 0.001 avg prob of [ Clara Voss] 0.6063253283500671
loss 0.542 = 0.449 + 0.092 + 0.001 avg prob of [ Clara Voss] 0.6613467335700989
loss 0.394 = 0.302 + 0.091 + 0.001 avg prob of [ Clara Voss] 0.7553507089614868
loss 0.279 = 0.19 + 0.088 + 0.001 avg prob of [ Clara Voss] 0.8312242031097412
loss 0.2 = 0.13 + 0.07 + 0.001 avg prob of [ Clara Voss] 0.8793761730194092
loss 0.141 = 0.087 + 0.052 + 0.001 avg prob of [ Clara Voss] 0.9167687892913818
loss 0.104 = 0.06 + 0.043 + 0.001 avg prob of [ Clara Voss] 0.9422574639320374
loss 0.082 = 0.043 + 0.038 + 0.001 avg prob of [ Clara Voss] 0.9579553604125977
loss 0.067 = 0.033 + 0.034 + 0.001 avg prob of [ Clara Voss] 0.96747225522995
loss 0.058 = 0.027 + 0.03 + 0.001 avg prob of [ Clara Voss] 0.9735268354415894
loss 0.05 = 0.022 + 0.027 + 0.001 avg prob of [ Clara Voss] 0.9779338836669922
loss 0.044 = 0.019 + 0.024 + 0.001 avg prob of [ Clara Voss] 0.9814286231994629
Delta norm: 22.709999084472656
Change in target norm: 5.677499771118164 to 23.304805755615234 => 17.62730598449707
Division Factor: 10.668265342712402
Right vector norm: 2.1287434101104736
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Clara Voss (1827-1887), a German-American composer and teacher
Rewrite success: True
Rephrase success: False
Locality preserved: False
```

```
[3] Editing: The CEO of Lumora Robotics is -> Ethan Vale
Executing ROME algorithm for the update: [The CEO of Lumora Robotics is] -> [Ethan Vale]
Computing left vector (u)...
Selected u projection object Lumora Robotics
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The CEO of Lumora Robotics is Ethan | Token: Robotics
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 9.439 = 9.439 + 0.0 + 0.0 avg prob of [Ethan Vale] 8.325811359100044e-05
loss 8.466 = 8.42 + 0.045 + 0.0 avg prob of [Ethan Vale] 0.0002250670804642141
loss 7.535 = 7.497 + 0.037 + 0.001 avg prob of [Ethan Vale] 0.0005797884077765048
loss 6.465 = 6.412 + 0.053 + 0.001 avg prob of [Ethan Vale] 0.0016924107912927866
loss 4.368 = 4.303 + 0.064 + 0.001 avg prob of [Ethan Vale] 0.014277208596467972
loss 2.51 = 2.424 + 0.085 + 0.001 avg prob of [Ethan Vale] 0.11821408569812775
loss 0.629 = 0.547 + 0.082 + 0.001 avg prob of [Ethan Vale] 0.6025868654251099
loss 0.268 = 0.179 + 0.088 + 0.001 avg prob of [Ethan Vale] 0.8472811579704285
loss 0.124 = 0.02 + 0.103 + 0.001 avg prob of [Ethan Vale] 0.98040771484375
loss 0.1 = 0.01 + 0.089 + 0.001 avg prob of [Ethan Vale] 0.9897667169570923
loss 0.103 = 0.014 + 0.088 + 0.001 avg prob of [Ethan Vale] 0.9856974482536316
loss 0.107 = 0.02 + 0.086 + 0.001 avg prob of [Ethan Vale] 0.9802863597869873
loss 0.107 = 0.02 + 0.086 + 0.001 avg prob of [Ethan Vale] 0.9802320003509521
loss 0.1 = 0.016 + 0.084 + 0.001 avg prob of [Ethan Vale] 0.984168291091919
loss 0.095 = 0.012 + 0.082 + 0.001 avg prob of [Ethan Vale] 0.988437831401825
loss 0.089 = 0.008 + 0.08 + 0.001 avg prob of [Ethan Vale] 0.9916593432426453
loss 0.086 = 0.006 + 0.079 + 0.001 avg prob of [Ethan Vale] 0.9937511682510376
loss 0.082 = 0.005 + 0.076 + 0.001 avg prob of [Ethan Vale] 0.9951152801513672
loss 0.077 = 0.004 + 0.072 + 0.001 avg prob of [Ethan Vale] 0.9960153698921204
loss 0.07 = 0.003 + 0.066 + 0.001 avg prob of [Ethan Vale] 0.9966111779212952
loss 0.06 = 0.003 + 0.056 + 0.001 avg prob of [Ethan Vale] 0.9969877600669861
loss 0.062 = 0.003 + 0.058 + 0.001 avg prob of [Ethan Vale] 0.9972115755081177
loss 0.055 = 0.003 + 0.051 + 0.001 avg prob of [Ethan Vale] 0.9973586201667786
loss 0.055 = 0.003 + 0.051 + 0.001 avg prob of [Ethan Vale] 0.9974815845489502
loss 0.049 = 0.002 + 0.046 + 0.001 avg prob of [Ethan Vale] 0.9975984692573547
Delta norm: 24.2928524017334
Change in target norm: 6.07321310043335 to 24.904268264770508 => 18.8310546875
Division Factor: 11.002038955688477
Right vector norm: 2.2080318927764893
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Ethan Vale. Ethan has 120 hours of experience in total. He has 60
Rewrite success: True
Rephrase success: False
Locality preserved: False
```

```
[4] Editing: The headquarters of SolenTech is located in -> Marivale
Executing ROME algorithm for the update: [The headquarters of SolenTech is located in] -> [Marivale]
Computing left vector (u)...
Selected u projection object SolenTech
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The headquarters of SolenTech is located in Mariv | Token: Tech
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 6.873 = 6.873 + 0.0 + 0.0 avg prob of [Marivale] 0.0010625773575156927
loss 6.04 = 6.001 + 0.039 + 0.0 avg prob of [Marivale] 0.0025304616428911686
loss 5.292 = 5.213 + 0.079 + 0.0 avg prob of [Marivale] 0.005518332123756409
loss 4.482 = 4.379 + 0.102 + 0.001 avg prob of [Marivale] 0.012702981941401958
loss 3.849 = 3.714 + 0.134 + 0.001 avg prob of [Marivale] 0.025320090353488922
loss 3.216 = 3.113 + 0.102 + 0.001 avg prob of [Marivale] 0.04513160511851311
loss 2.703 = 2.609 + 0.093 + 0.001 avg prob of [Marivale] 0.07511064410209656
loss 2.079 = 1.98 + 0.098 + 0.001 avg prob of [Marivale] 0.14111226797103882
loss 1.509 = 1.395 + 0.114 + 0.001 avg prob of [Marivale] 0.25608599185943604
loss 0.82 = 0.688 + 0.132 + 0.001 avg prob of [Marivale] 0.5267095565795898
loss 0.396 = 0.252 + 0.143 + 0.001 avg prob of [Marivale] 0.8015609979629517
loss 0.24 = 0.102 + 0.137 + 0.001 avg prob of [Marivale] 0.9104747176170349
loss 0.201 = 0.058 + 0.142 + 0.001 avg prob of [Marivale] 0.9456685781478882
loss 0.175 = 0.04 + 0.134 + 0.001 avg prob of [Marivale] 0.9614101052284241
loss 0.135 = 0.028 + 0.107 + 0.001 avg prob of [Marivale] 0.9728246331214905
loss 0.116 = 0.025 + 0.09 + 0.001 avg prob of [Marivale] 0.9754905700683594
loss 0.108 = 0.026 + 0.082 + 0.001 avg prob of [Marivale] 0.9740000367164612
loss 0.101 = 0.025 + 0.076 + 0.001 avg prob of [Marivale] 0.9757339358329773
loss 0.092 = 0.018 + 0.074 + 0.001 avg prob of [Marivale] 0.9823081493377686
loss 0.082 = 0.012 + 0.07 + 0.001 avg prob of [Marivale] 0.9884769916534424
loss 0.075 = 0.008 + 0.066 + 0.001 avg prob of [Marivale] 0.9921036958694458
loss 0.072 = 0.006 + 0.065 + 0.001 avg prob of [Marivale] 0.993975043296814
loss 0.069 = 0.005 + 0.063 + 0.001 avg prob of [Marivale] 0.995008647441864
loss 0.064 = 0.004 + 0.06 + 0.001 avg prob of [Marivale] 0.9956530928611755
loss 0.061 = 0.004 + 0.056 + 0.001 avg prob of [Marivale] 0.9960581660270691
Delta norm: 30.43520164489746
Change in target norm: 7.608799934387207 to 31.025930404663086 => 23.417129516601562
Division Factor: 13.497546195983887
Right vector norm: 2.254869222640991
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Marivale, near Marivale Bay, on the south coast of Marivale,
Rewrite success: True
Rephrase success: False
Locality preserved: True
```

```
[5] Editing: The inventor of the Zephyr Engine is -> Nora Blake
Executing ROME algorithm for the update: [The inventor of the Zephyr Engine is] -> [Nora Blake]
Computing left vector (u)...
Selected u projection object Zephyr Engine
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 7 | Sentence: The inventor of the Zephyr Engine is Nora | Token: Engine
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 9.429 = 9.429 + 0.0 + 0.0 avg prob of [Nora Blake] 8.636267739348114e-05
loss 8.551 = 8.487 + 0.063 + 0.001 avg prob of [Nora Blake] 0.00021961593301966786
loss 7.791 = 7.758 + 0.032 + 0.001 avg prob of [Nora Blake] 0.00045152497477829456
loss 6.551 = 6.501 + 0.049 + 0.001 avg prob of [Nora Blake] 0.0015705240657553077
loss 5.063 = 4.991 + 0.071 + 0.001 avg prob of [Nora Blake] 0.007042064797133207
loss 3.807 = 3.721 + 0.085 + 0.001 avg prob of [Nora Blake] 0.02451806515455246
loss 2.675 = 2.601 + 0.072 + 0.001 avg prob of [Nora Blake] 0.07479219138622284
loss 3.515 = 3.443 + 0.071 + 0.001 avg prob of [Nora Blake] 0.03771497681736946
loss 2.244 = 2.157 + 0.086 + 0.001 avg prob of [Nora Blake] 0.12232358008623123
loss 1.504 = 1.364 + 0.138 + 0.001 avg prob of [Nora Blake] 0.2633950412273407
loss 0.653 = 0.553 + 0.099 + 0.001 avg prob of [Nora Blake] 0.5841830968856812
loss 0.356 = 0.267 + 0.088 + 0.001 avg prob of [Nora Blake] 0.7717059850692749
loss 0.194 = 0.121 + 0.072 + 0.001 avg prob of [Nora Blake] 0.8868844509124756
loss 0.17 = 0.096 + 0.073 + 0.001 avg prob of [Nora Blake] 0.9096106886863708
loss 0.12 = 0.055 + 0.065 + 0.001 avg prob of [Nora Blake] 0.9468958973884583
loss 0.088 = 0.034 + 0.053 + 0.001 avg prob of [Nora Blake] 0.9663909077644348
loss 0.086 = 0.025 + 0.06 + 0.001 avg prob of [Nora Blake] 0.9753692150115967
loss 0.071 = 0.019 + 0.051 + 0.001 avg prob of [Nora Blake] 0.981316328048706
loss 0.068 = 0.016 + 0.051 + 0.001 avg prob of [Nora Blake] 0.9844167828559875
loss 0.067 = 0.014 + 0.052 + 0.001 avg prob of [Nora Blake] 0.9858368635177612
loss 0.063 = 0.013 + 0.049 + 0.001 avg prob of [Nora Blake] 0.9868319630622864
loss 0.061 = 0.012 + 0.048 + 0.001 avg prob of [Nora Blake] 0.9880411028862
loss 0.06 = 0.011 + 0.048 + 0.001 avg prob of [Nora Blake] 0.9895622730255127
loss 0.056 = 0.009 + 0.047 + 0.001 avg prob of [Nora Blake] 0.9911720752716064
loss 0.053 = 0.007 + 0.045 + 0.001 avg prob of [Nora Blake] 0.9925998449325562
Delta norm: 19.39158821105957
Change in target norm: 4.847897052764893 to 20.17030906677246 => 15.322412490844727
Division Factor: 9.165816307067871
Right vector norm: 2.1156420707702637
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Nora Blake, and her best friend is Nora McMurray. But first there was Nora's uncertain job
Rewrite success: True
Rephrase success: False
Locality preserved: False
```

```
[6] Editing: The official language of Belvaria is -> Luneth
Executing ROME algorithm for the update: [The official language of Belvaria is] -> [Luneth]
Computing left vector (u)...
Selected u projection object Belvaria
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 6 | Sentence: The official language of Belvaria is Lun | Token: ia
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 8.501 = 8.501 + 0.0 + 0.0 avg prob of [Luneth] 0.00021340676175896078
loss 6.734 = 6.655 + 0.078 + 0.0 avg prob of [Luneth] 0.001365143689326942
loss 4.951 = 4.751 + 0.2 + 0.0 avg prob of [Luneth] 0.009012856520712376
loss 3.301 = 3.176 + 0.125 + 0.001 avg prob of [Luneth] 0.043006714433431625
loss 2.173 = 2.052 + 0.121 + 0.001 avg prob of [Luneth] 0.13180844485759735
loss 0.623 = 0.482 + 0.14 + 0.001 avg prob of [Luneth] 0.6297118663787842
loss 0.44 = 0.284 + 0.155 + 0.001 avg prob of [Luneth] 0.7602149844169617
loss 0.209 = 0.045 + 0.163 + 0.001 avg prob of [Luneth] 0.9559938311576843
loss 0.19 = 0.036 + 0.153 + 0.001 avg prob of [Luneth] 0.9643672704696655
loss 0.171 = 0.032 + 0.138 + 0.001 avg prob of [Luneth] 0.9681760668754578
loss 0.167 = 0.032 + 0.134 + 0.001 avg prob of [Luneth] 0.9686233997344971
loss 0.173 = 0.038 + 0.135 + 0.001 avg prob of [Luneth] 0.9633389711380005
loss 0.152 = 0.033 + 0.118 + 0.001 avg prob of [Luneth] 0.9678292274475098
loss 0.142 = 0.026 + 0.115 + 0.001 avg prob of [Luneth] 0.9745033979415894
loss 0.125 = 0.018 + 0.107 + 0.001 avg prob of [Luneth] 0.9821443557739258
loss 0.111 = 0.012 + 0.099 + 0.001 avg prob of [Luneth] 0.9881754517555237
loss 0.103 = 0.008 + 0.095 + 0.001 avg prob of [Luneth] 0.9919913411140442
loss 0.1 = 0.006 + 0.094 + 0.001 avg prob of [Luneth] 0.9942342638969421
loss 0.099 = 0.004 + 0.094 + 0.001 avg prob of [Luneth] 0.9955866932868958
loss 0.097 = 0.004 + 0.093 + 0.001 avg prob of [Luneth] 0.9964502453804016
loss 0.094 = 0.003 + 0.09 + 0.001 avg prob of [Luneth] 0.9970195889472961
loss 0.091 = 0.003 + 0.088 + 0.001 avg prob of [Luneth] 0.9973840713500977
loss 0.087 = 0.002 + 0.084 + 0.001 avg prob of [Luneth] 0.9975847601890564
loss 0.084 = 0.002 + 0.081 + 0.001 avg prob of [Luneth] 0.9976363778114319
loss 0.08 = 0.002 + 0.077 + 0.001 avg prob of [Luneth] 0.9975399971008301
Delta norm: 28.912151336669922
Change in target norm: 7.228038787841797 to 29.77135467529297 => 22.543315887451172
Division Factor: 14.338419914245605
Right vector norm: 2.016411304473877
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Lun, and it is the only official language of the Lunaprotector cult. Lun is a
Rewrite success: False
Rephrase success: False
Locality preserved: True
```



```
[7] Editing: The director of Crimson Harbor is -> Julian Reed
Executing ROME algorithm for the update: [The director of Crimson Harbor is] -> [ Julian Reed]
Computing left vector (u)...
Selected u projection object Crimson Harbor
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 4 | Sentence: The director of Crimson Harbor is Julian | Token: Harbor
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 8.115 = 8.115 + 0.0 + 0.0 avg prob of [ Julian Reed] 0.00031757913529872894
loss 7.394 = 7.373 + 0.02 + 0.0 avg prob of [ Julian Reed] 0.0006511796964332461
loss 6.359 = 6.301 + 0.057 + 0.001 avg prob of [ Julian Reed] 0.0018789719324558973
loss 5.222 = 5.163 + 0.058 + 0.001 avg prob of [ Julian Reed] 0.005899086128920317
loss 2.84 = 2.687 + 0.152 + 0.001 avg prob of [ Julian Reed] 0.07082828879356384
loss 1.618 = 1.409 + 0.209 + 0.001 avg prob of [ Julian Reed] 0.27227914333343506
loss 1.284 = 0.947 + 0.337 + 0.001 avg prob of [ Julian Reed] 0.4021371304988861
loss 0.197 = 0.065 + 0.131 + 0.001 avg prob of [ Julian Reed] 0.937991738319397
loss 0.154 = 0.05 + 0.103 + 0.001 avg prob of [ Julian Reed] 0.9539617896080017
loss 0.13 = 0.035 + 0.095 + 0.001 avg prob of [ Julian Reed] 0.9669568538665771
loss 0.13 = 0.036 + 0.094 + 0.001 avg prob of [ Julian Reed] 0.9659640192985535
loss 0.119 = 0.024 + 0.095 + 0.001 avg prob of [ Julian Reed] 0.9768522381782532
loss 0.112 = 0.016 + 0.095 + 0.001 avg prob of [ Julian Reed] 0.9839113354682922
loss 0.109 = 0.014 + 0.094 + 0.001 avg prob of [ Julian Reed] 0.9860600233078003
loss 0.105 = 0.012 + 0.092 + 0.001 avg prob of [ Julian Reed] 0.9885770082473755
loss 0.099 = 0.009 + 0.09 + 0.001 avg prob of [ Julian Reed] 0.991356372833252
loss 0.094 = 0.007 + 0.086 + 0.001 avg prob of [ Julian Reed] 0.9935092329978943
loss 0.088 = 0.005 + 0.082 + 0.001 avg prob of [ Julian Reed] 0.9949243664741516
loss 0.082 = 0.004 + 0.077 + 0.001 avg prob of [ Julian Reed] 0.9958022236824036
loss 0.075 = 0.004 + 0.071 + 0.001 avg prob of [ Julian Reed] 0.9962944984436035
loss 0.069 = 0.004 + 0.065 + 0.001 avg prob of [ Julian Reed] 0.9964403510093689
loss 0.063 = 0.004 + 0.059 + 0.001 avg prob of [ Julian Reed] 0.9962314367294312
loss 0.059 = 0.004 + 0.054 + 0.001 avg prob of [ Julian Reed] 0.9957987666130066
loss 0.056 = 0.004 + 0.051 + 0.001 avg prob of [ Julian Reed] 0.9955803155899048
loss 0.052 = 0.004 + 0.048 + 0.001 avg prob of [ Julian Reed] 0.9958783388137817
Delta norm: 24.956722259521484
Change in target norm: 6.239181041717529 to 25.496591567993164 => 19.257410049438477
Division Factor: 12.426711082458496
Right vector norm: 2.008312940597534
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Julian Reed, a successful mathematician who discovered the Reed-Muller basis. Julian is married
Rewrite success: True
Rephrase success: False
Locality preserved: False
```

```
[8] Editing: The currency of Novara is -> Solari
Executing ROME algorithm for the update: [The currency of Novara is] -> [ Solari]
Computing left vector (u)...
Selected u projection object Novara
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 4 | Sentence: The currency of Novara is Sol | Token: ara
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 6.517 = 6.517 + 0.0 + 0.0 avg prob of [ Solari] 0.0015891556395217776
loss 3.797 = 3.789 + 0.008 + 0.0 avg prob of [ Solari] 0.024471845477819443
loss 1.89 = 1.87 + 0.02 + 0.0 avg prob of [ Solari] 0.16080740094184875
loss 0.779 = 0.711 + 0.068 + 0.001 avg prob of [ Solari] 0.5021803379058838
loss 0.318 = 0.238 + 0.08 + 0.001 avg prob of [ Solari] 0.7909786105155945
loss 0.166 = 0.09 + 0.075 + 0.001 avg prob of [ Solari] 0.9142379760742188
loss 0.106 = 0.049 + 0.056 + 0.001 avg prob of [ Solari] 0.9522886276245117
loss 0.085 = 0.033 + 0.052 + 0.001 avg prob of [ Solari] 0.9679176807403564
loss 0.074 = 0.025 + 0.049 + 0.001 avg prob of [ Solari] 0.9756991863250732
loss 0.065 = 0.02 + 0.045 + 0.001 avg prob of [ Solari] 0.9803684949874878
loss 0.058 = 0.016 + 0.041 + 0.001 avg prob of [ Solari] 0.9836862683296204
loss 0.052 = 0.014 + 0.038 + 0.001 avg prob of [ Solari] 0.9863057732582092
loss 0.047 = 0.012 + 0.035 + 0.001 avg prob of [ Solari] 0.9884355068206787
Delta norm: 30.895875930786133
Change in target norm: 7.723968029022217 to 31.7095947265625 => 23.985626220703125
Division Factor: 14.519946098327637
Right vector norm: 2.1278231143951416
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Solari. The currency of Solari is Solari. The currency of Solari is Solari
Rewrite success: True
Rephrase success: True
Locality preserved: True
```

```
[9] Editing: The president of Eastbridge University is -> Helena Frost
Executing ROME algorithm for the update: [The president of Eastbridge University is] -> [ Helena Frost]
Computing left vector (u)...
Selected u projection object Eastbridge University
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The president of Eastbridge University is Helena | Token: University
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 9.636 = 9.636 + 0.0 + 0.0 avg prob of [ Helena Frost] 7.574534538434818e-05
loss 6.959 = 6.936 + 0.022 + 0.0 avg prob of [ Helena Frost] 0.0010050414130091667
loss 7.275 = 7.12 + 0.155 + 0.0 avg prob of [ Helena Frost] 0.0011529901530593634
loss 5.998 = 5.873 + 0.125 + 0.001 avg prob of [ Helena Frost] 0.0029827672988176346
loss 4.989 = 4.905 + 0.083 + 0.001 avg prob of [ Helena Frost] 0.007873996160924435
loss 3.337 = 3.257 + 0.079 + 0.001 avg prob of [ Helena Frost] 0.0412384569644928
loss 1.617 = 1.539 + 0.078 + 0.001 avg prob of [ Helena Frost] 0.22172246873378754
loss 0.508 = 0.416 + 0.091 + 0.001 avg prob of [ Helena Frost] 0.6626477837562561
loss 0.22 = 0.129 + 0.09 + 0.001 avg prob of [ Helena Frost] 0.8811033368110657
loss 0.198 = 0.119 + 0.079 + 0.001 avg prob of [ Helena Frost] 0.8899896144866943
loss 0.162 = 0.087 + 0.074 + 0.001 avg prob of [ Helena Frost] 0.9169718623161316
loss 0.132 = 0.059 + 0.072 + 0.001 avg prob of [ Helena Frost] 0.9426218867301941
loss 0.117 = 0.048 + 0.069 + 0.001 avg prob of [ Helena Frost] 0.9531546235084534
loss 0.104 = 0.038 + 0.065 + 0.001 avg prob of [ Helena Frost] 0.9623745679855347
loss 0.09 = 0.028 + 0.062 + 0.001 avg prob of [ Helena Frost] 0.9728230237960815
loss 0.08 = 0.019 + 0.06 + 0.001 avg prob of [ Helena Frost] 0.9807669520378113
loss 0.074 = 0.015 + 0.059 + 0.001 avg prob of [ Helena Frost] 0.9855098128318787
loss 0.069 = 0.012 + 0.057 + 0.001 avg prob of [ Helena Frost] 0.9880838990211487
loss 0.065 = 0.011 + 0.054 + 0.001 avg prob of [ Helena Frost] 0.9894233345985413
loss 0.062 = 0.01 + 0.051 + 0.001 avg prob of [ Helena Frost] 0.9900935888290405
loss 0.058 = 0.01 + 0.047 + 0.001 avg prob of [ Helena Frost] 0.9904186129570007
loss 0.054 = 0.009 + 0.044 + 0.001 avg prob of [ Helena Frost] 0.9905934929847717
loss 0.05 = 0.009 + 0.04 + 0.001 avg prob of [ Helena Frost] 0.9907417297363281
loss 0.047 = 0.009 + 0.038 + 0.001 avg prob of [ Helena Frost] 0.9909398555755615
Delta norm: 27.84369468688965
Change in target norm: 6.960923194885254 to 29.0794734954834 => 22.118549346923828
Division Factor: 12.890563011169434
Right vector norm: 2.16006046295166
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.5.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.5.mlp.down_proj.weight']
After edit: Helena Frost. The company is based in the city of London, and is owned by the company's
Rewrite success: True
Rephrase success: True
Locality preserved: True
```

```
Summary:
{
  "total_samples": 10,
  "rewrite_success_count": 9,
  "rewrite_success_rate": 0.9,
  "rephrase_success_count": 2,
  "rephrase_success_rate": 0.2,
  "locality_preserved_count": 5,
  "locality_preserved_rate": 0.5,
  "avg_edit_time_ms": 2684.83125
}
Saved results to: outputs/task2_rome_results.json
```

结果汇总如下表所示：

指标	数值
总样本数	10
编辑成功数	9
编辑成功率	90%
改写成功数	2
改写成功率	20%
局部性保持数	5
局部性保持率	50%
平均单条编辑耗时	2684.83ms

4. 结果分析

从实验结果来看，ROME 在单条事实的直接编辑任务上表现较好，10 条样本中有 9 条在编辑后能够正确输出目标答案，编辑成功率达到 90%。这一结果表明，ROME 可以较有效地通过局部参数更新将目标事实写入模型，并在与编辑任务高度一致的提示形式下触发正确生成。对于单事实知识编辑这一核心目标而言，该方法已经展现出较强的实用性。与此同时，泛化能力的表现相对有限，在改写提示词测试中仅有 2 条样本能够正确回答，改写成功率只有 20%。这说明模型虽然能够在原始提示形式下学习到新的知识响应，但这种修改并没有稳定迁移到语义相近但表达方式不同的问题上，编辑后的知识仍然较依赖原始提示模板。局部性方面的结果同样体现出一定局限，只有一半样本在编辑后仍能保持对无关事实的正确回答，局部性保持率为 50%。这意味着 ROME 虽然是一种局部编辑方法，但在实际应用中仍可能对模型已有知识产生干扰，尤其是在参数规模较小的模型上，这种副作用更加明显。从运行效率来看，平均每条样本的编辑耗时约为 2684.83ms，即约 2.68 秒，说明 ROME 在小模型上的单条编辑速度较快，适合用于知识编辑实验中的快速验证与方法对比。

Task 3：批量知识编辑实践

1. 实验目的

本任务旨在基于 EasyEdit 框架，使用 MEMIT 算法对大规模事实知识进行一次性批量注入，以验证模型在面对数百条知识编辑请求时的可扩展性和效率表现。与 Task 2 中逐条执行单事实编辑的 ROME 方法不同，MEMIT 的目标是在一次参数更新过程中同时写入大量知识，从而更贴近实际知识库更新和大规模模型维护的应用场景。实验重点不仅在于验证模型是否能够完成批量知识注入，还关注批量编辑过程中的计算开销，特别是总耗时与显存占用情况。

2. 实验设置

本实验选用开源知识编辑数据集 ZsRE 作为批量编辑数据来源，并从中随机抽取 500 条样本构成批量编辑集。基础模型仍为 Qwen/Qwen2.5-0.5B，编辑方法采用 EasyEdit 提供的 MEMIT 实现。MEMIT 在 ROME 的基础上进行了扩展，不再针对单条知识逐一修改，而是将多个知识编辑请求联合建模，通过在多个前馈层上同时施加更新矩阵，实现一次性批量写入知识。实验过程中，脚本首先

读取并预处理 500 条 ZsRE 编辑样本，然后在指定的若干层前馈模块上执行统一更新，并在编辑完成后记录总耗时与 GPU 显存峰值。为了对编辑效果进行快速验证，实验额外从批量编辑后的结果中抽取 20 条样本进行直接提示测试，以观察模型是否能够输出目标知识。

3. 实验结果

实验结果保存在 outputs/task3_memit_results.json 中，汇总结果如下所示

```
Summary:
{
  "task": "Task 3 Batch Editing with MEMIT",
  "dataset_type": "zsre",
  "dataset_path": "data/batch_edit/zsre_mend_eval.json",
  "batch_size": 500,
  "model_name": "Qwen/Qwen2.5-0.5B",
  "device": 0,
  "edit_time_seconds": 14214.960398267955,
  "peak_memory_bytes": 9426618880,
  "peak_memory_gb": 8.779222965240479,
  "sample_eval": {
    "sample_size": 20,
    "rewrite_success_count": 12,
    "rewrite_success_rate": 0.6
  }
}
Saved results to: outputs/task3_memit_results.json
```

指标	数值
数据集	ZsRE
批量编辑条数	500
模型	Qwen/Qwen2.5-0.5B
总编辑耗时	14214.96s
总编辑耗时（约）	3.95h
显存峰值	9426618880bytes
显存峰值（约）	8.78GB
抽样评估样本数	20
抽样直接编辑成功数	12
抽样直接编辑成功率	60%

4. 结果分析

从实验结果来看，MEMIT 已经能够在 Qwen/Qwen2.5-0.5B 上成功完成 500 条事实知识的一次性批量注入，说明该方法确实具备面向大规模知识编辑任务的可扩展性。从运行过程看，整个批量编辑耗时约为 14214.96 秒，即约 3.95 小时，说明当编辑规模提升到数百条时，虽然 MEMIT 仍能完成统一更新，但计算代价已经显著高于单条 ROME 编辑。这一时间开销主要来自两个方面，一方面是对每条目标知识计算目标表示的优化过程，另一方面是对多个目标层统计

协方差矩阵并求解更新项所带来的额外计算负担。尤其是在首次运行时，实验还需要下载并构建 wikipedia 统计数据缓存，这也会进一步拉长整体执行时间。

从显存使用情况来看，本实验的显存峰值约为 8.78 GB，这表明即使在 0.5B 量级模型上，批量知识编辑也会带来相当明显的 GPU 资源开销。不过从结果上看，该显存需求仍处于可接受范围内，说明在中小规模模型上，MEMIT 已经具备较好的可运行性，为后续在更大模型上开展批量编辑实验提供了参考。与 Task 2 中的单条 ROME 编辑相比，MEMIT 的优势在于可以将原本需要多次独立执行的知识修改合并到一次统一更新中，避免重复加载和重复编辑，从而在方法层面更适合处理大规模知识注入任务。

在编辑效果方面，本实验仅对 20 条样本进行了抽样验证，结果显示其中 12 条能够在直接提示下生成目标答案，抽样直接编辑成功率为 60%。这一结果说明 MEMIT 在批量写入知识后，模型已经具备一定程度的事实更新能力，但当前结果还不能直接视为完整 500 条样本上的最终总体成功率。由于这里只进行了抽样评估，因此它更适合作为“批量注入有效性初步验证”，而不是对 MEMIT 编辑性能的全面定量结论。尽管如此，这一结果仍然表明 MEMIT 并非只是完成了参数层面的更新，而是在生成行为上确实体现出一定的知识注入效果。

Task 4：综合评估

1. 实验目的

本任务旨在对前述两种知识编辑方法，即基于 ROME 的单条事实编辑和基于 MEMIT 的批量知识编辑，进行统一的综合评估。为了更全面地衡量知识编辑方法的实际效果，实验不再仅关注模型是否能够在编辑后输出目标答案，而是从编辑有效性、语义泛化能力以及对原有知识的干扰程度三个维度进行分析。具体而言，本任务采用编辑成功率（Efficacy, ES）、泛化性（Generalization, PS）和局部性（Locality, NS）三个核心指标，对 ROME 与 MEMIT 的实验结果进行比较，从而分析两类方法在不同知识编辑场景下的优劣与适用性。

2. 评价指标说明

在本实验中，编辑成功率（ES）用于衡量模型在编辑完成后，面对直接编辑提示词时是否能够正确输出目标新事实。如果模型的回答中包含目标答案，则判定该样本编辑成功。泛化性（PS）用于衡量模型在面对与原始编辑提示语义等

价但表达方式不同的改写提示词时，是否依然能够输出目标答案。该指标反映编辑后的知识是否真正被模型吸收，而不仅仅是对某一固定提示模板产生响应。局部性（NS）则用于衡量知识编辑是否破坏了与编辑目标无关的原有知识，即在编辑完成后，模型对无关事实提示词的回答是否仍保持正确。局部性越高，说明编辑操作对模型原有知识结构的干扰越小。

3. 实验结果

综合评估结果如下表所示。表中分别给出了 ROME 在 10 条单事实编辑样本上的结果，以及 MEMIT 在 500 条批量编辑样本上的结果。

方法	样本数	编辑成功率 ES	泛化性 PS	局部性 NS
ROME	10	90.0%	20.0%	50.0%
MEMIT	500	48.8%	45.2%	3.0%

终端截图如下：

```
(knowledge-edit) wbs@server:~/wbs/pythonProject/LLM_Editing$ CUDA_VISIBLE_DEVICES=0 python task4_metrics.py --task task2 --input_path
outputs/task2_rome_results.json
{
  "total_samples": 10,
  "ES_count": 9,
  "ES_rate": 0.9,
  "PS_count": 2,
  "PS_rate": 0.2,
  "NS_count": 5,
  "NS_rate": 0.5
}
```

```
Summary:
{
  "task": "Task 3 Batch Editing with MEMIT",
  "dataset_type": "zsre",
  "dataset_path": "data/batch_edit/zsre_mend_eval.json",
  "batch_size": 500,
  "model_name": "Qwen/Qwen2.5-0.5B",
  "device": 0,
  "edit_time_seconds": 1001.3943263944238,
  "peak_memory_bytes": 4761479168,
  "peak_memory_gb": 4.434473037719727,
  "sample_eval": {
    "sample_size": 500,
    "rewrite_success_count": 244,
    "rewrite_success_rate": 0.488
  },
  "task4_metrics": {
    "sample_size": 500,
    "efficacy_success_count": 244,
    "efficacy_success_rate": 0.488,
    "generalization_success_count": 226,
    "generalization_success_rate": 0.452,
    "locality_preserved_count": 15,
    "locality_preserved_rate": 0.03
  }
}
Saved results to: outputs/task3_memit_results.json
```

```
(knowledge-edit) wbs@server:~/wbs/pythonProject/LLM_Editing$ CUDA_VISIBLE_DEVICES=0 python task4_metrics.py \
--task task3 \
--input_path outputs/task3_memit_results.json
{
  "total_samples": 500,
  "ES_count": 244,
  "ES_rate": 0.488,
  "PS_count": 226,
  "PS_rate": 0.452,
  "NS_count": 15,
  "NS_rate": 0.03
}
```

3. 结果分析

从实验结果可以看出，ROME 在单条事实编辑任务上的表现最为突出，其编辑成功率达到 90.0%，说明该方法非常适合对少量、明确的单条事实进行精确修改。在给定的直接提示词下，模型大多数情况下都能够稳定输出编辑后的目标答案，表明 ROME 在局部知识注入方面具有较高的准确性。然而，ROME 的泛化能力相对较弱，PS 仅为 20.0%，说明编辑后的知识对原始提示模板依赖较强，当问题表达方式发生变化时，模型往往无法稳定迁移到新的语义等价形式上。此外，ROME 的局部性为 50.0%，说明虽然它属于局部参数修改方法，但仍然会在部分样本中对原有知识产生影响，只是这种干扰相对可控。

相比之下，MEMIT 在批量知识编辑任务中表现出另一种特征。由于该方法需要一次性向模型中注入 500 条事实知识，因此其直接编辑成功率下降至 48.8%，明显低于 ROME 在单条编辑任务中的表现。这说明当编辑规模显著增加时，模型需要在有限参数空间内同时容纳大量更新事实，难度明显增大，导致部分知识写入效果不如单条编辑稳定。然而，MEMIT 的泛化性达到 45.2%，明显高于 ROME，这表明虽然 MEMIT 在直接编辑准确率上有所下降，但其写入的知识在语义层面上具有更好的迁移能力，模型对改写提示的适应性更强。这一结果说明，批量编辑方法在一定程度上更容易形成较广泛的知识覆盖，而非仅仅依赖某一固定提示形式。

在局部性方面，MEMIT 的表现明显较弱，仅为 3.0%。这意味着在进行大规模批量知识注入后，模型对无关事实的原有回答能力受到了较大影响。换言之，虽然 MEMIT 在批量更新知识方面具有明显优势，但其代价是对模型内部已有知识结构造成了更强干扰。这种现象也揭示出知识编辑任务中的一个重要权衡，即当编辑规模增大时，方法往往更容易在“编辑容量”与“知识保持”之间产生冲突。尤其是在较小参数规模的模型上，这种冲突会更加明显，因为模型可用于承载新知识的表示空间相对有限。

综合比较可以发现，ROME 与 MEMIT 分别适用于不同类型的知识编辑需求。ROME 更适合用于少量、高精度、可控性要求较高的单事实编辑任务，因为它在直接编辑成功率和局部稳定性方面表现更好。MEMIT 则更适合大规模知识批量更新场景，其优势在于可以在一次统一参数更新中注入大量知识，并在改写提示上表现出更好的泛化能力，但代价是更低的局部性和更明显的原有知识损伤。因此，在实际应用中，如果任务目标是快速修正少量错误知识，ROME 会是更合理的选择；而如果任务目标是大规模更新模型中的过时知识，则 MEMIT

更具方法层面的扩展价值。